



UNIVERSIDAD DE GUADALAJARA

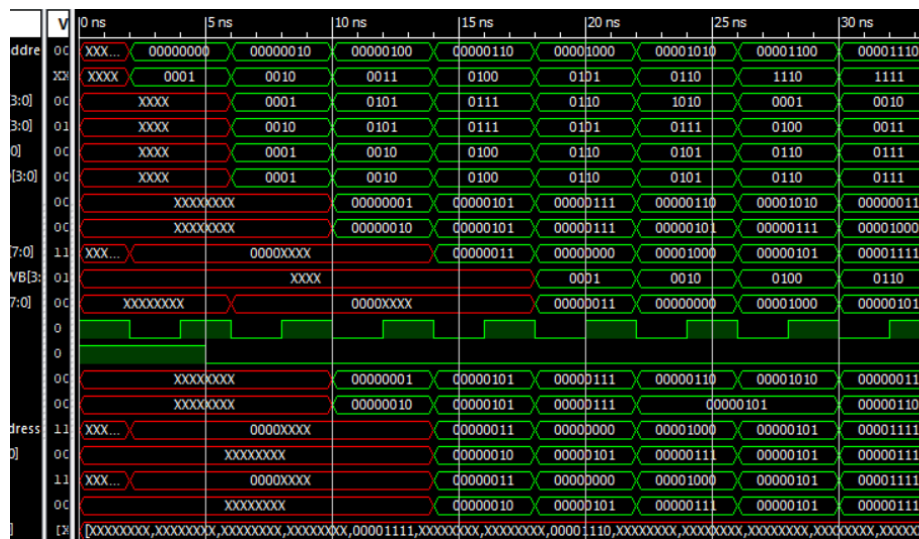
CUCEI



Carrera: Ingeniería en Computación
Materia: Arquitectura de Computadoras

Reporte

PROYECTO FINAL - FASE 2



Integrantes:

Hernández Zúñiga Roberto López López Karla Valeria
Ortiz Segura Emmanuel Alejandro Segura Gutierrez Aaron Ricardo

Maestro: López Arce Delgado Jorge Ernesto

Sección: D03

Fecha: 16 mayo 2026

Índice

1. Introducción

1.1 Procesador MIPS

1.2 Tipos de Instrucciones

1.3 Instrucciones a Implementar

2. Objetivos

2.1 Objetivo General

2.2 Objetivos Específicos

3. Marco Teórico

3.1 Instrucciones Tipo R

3.2 Operador Ternario

3.3 Desglose de Instrucciones Elegidas

3.4 Proceso de Compilación

3.5 Algoritmos Implementables con ISA

3.5 Búsqueda Binaria (Algoritmo)

3.6 Instrucciones Tipo I

3.7 Instrucciones Tipo J

4. Desarrollo

4.1 Pseudocódigo Lenguaje Ensamblador

4.2 Módulos Modificados

4.3 Ensamblador, script Python y .bin

4.4 Resultados Simulación

5. Conclusión

6. Bibliografía

1. Introducción

1.1 Procesador MIPS

En términos sencillos, el procesador MIPS es como un "minimalista" de la tecnología, la esencia del diseño RISC. Su filosofía se basa en que menos es más, al reducir la complejidad de las tareas, el chip puede trabajar a una mayor velocidad.

Puntos clave del diseño

Gestión de Memoria Selectiva:

A diferencia de otros procesadores que se distraen buscando datos en la memoria RAM constantemente, MIPS es muy estricto. Solo usa dos comandos específicos para mover datos (lw-Load word y sw-Store word). Todo el "trabajo sucio" de cálculos ocurre internamente en sus 32 registros, lo que evita cuellos de botella.

Instrucciones Uniformes:

Imaginemos que todas las herramientas en una caja tuvieran exactamente el mismo tamaño y peso. Eso es MIPS, con todas sus instrucciones de 32 bits. Esto permite que el procesador no pierda tiempo midiendo o analizando cada comando; simplemente los recibe y los ejecuta de forma rítmica.

Trabajo en Cadena (Pipelining):

Su estructura está diseñada para funcionar como una línea de ensamblaje industrial. Mientras una instrucción se está terminando de escribir, otra ya se está procesando y una nueva está entrando al procesador. No hay tiempos muertos.

El "Ancla" del Cero:

Tiene una característica curiosa pero brillante: el registro cero. Es un valor constante que nunca cambia, manteniendo siempre un valor de 0, lo que sirve como punto de referencia rápido para inicializar valores, mover y limpiar datos o realizar comparaciones lógicas sin esfuerzo extra.

Las etapas clásicas de un procesador MIPS son:

1. **IF (Instruction Fetch):** Traer la instrucción de memoria.
2. **ID (Instruction Decode):** Decodificar y leer registros.
3. **EX (Execute):** Realizar la operación en la ALU (Unidad Aritmético Lógica).
4. **MEM (Memory Access):** Acceder a la RAM si es necesario.
5. **WB (Write Back):** Escribir el resultado en el registro.

1.2 Tipos de instrucciones

En el diseño de procesadores, existen 3 tipos de instrucciones que facilitan la decodificación en la UC, con formato de 32 bits, se describen de la siguiente forma:

Tipo R (registro) (implementado en esta práctica)

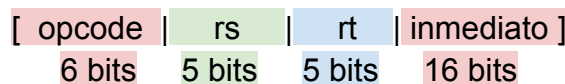
Para las operaciones aritmético-lógicas entre registros, por ejemplo, add y sub. Los 32 bits de la instrucción se dividen de la siguiente manera:



- **rs, rt:** Registros fuente.
- **rd:** Registro destino.
- **funct:** Código funcional que especifica la operación.
- Se decodifican los campos y se envían **rs** y **rt** a la ALU, el resultado de la operación especificada en **function** se escribe en la dirección de memoria **rd**.

Tipo I (inmediato)

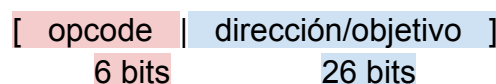
Para operaciones con operandos inmediatos (constantes), cargas/almacenamientos de memoria. Por ejemplo, sw (save word) y lw (load word). Su formato de 32 bits es el siguiente:



- **rs:** Registro fuente.
- **rt:** Registro destino o fuente (depende de la instrucción).
- **inmediato:** Valor constante de 16 bits, a menudo extendido con signo.
- El valor **inmediato** se extiende en signo y se utiliza junto con **rs** en la ALU.

Tipo J (salto/jump)

Para saltos incondicionales a direcciones de memoria. Por ejemplo, j(jump). El formato de instrucción a 32 bits es:



- **dirección:** Junto con el PC (Program Counter), define dirección de salto.

La unidad de control actualiza el PC directamente utilizando la **dirección** de 26 bits.

A continuación, se presenta el set de instrucciones que se utilizará en el proyecto.

1.3 Instrucciones a Implementar

Instrucción	Tipo	Sintaxis	Descripción
add 	R	add \$rd, \$rs, \$rt	Suma: $d = s + t$
sub 	R	sub \$rd, \$rs, \$rt	Resta: $d = s - t$
or 	R	or \$rd, \$rs, \$rt	Operación OR lógica: $d = s t$
and 	R	and \$rd, \$rs, \$rt	Operación AND lógica: $d = s \& t$
slt 	R	slt \$rd, \$rs, \$rt	Si $s < t$, $d = 1$; si no, $d = 0$
nop 	R	nop \$rd, \$rs, \$rt	No Operation: (equivale a sll \$0, \$0, 0)
addi 	I	addi \$rd, \$rs, constante	Suma inmediata: $d = s + \text{constante}$
subi 	I	addi \$rd, \$rs, -constante	Se usa addi con valor negativo
ori 	I	ori \$rd, \$rs, constante	OR inmediato: $d = s \text{constante}$
andi 	I	andi \$rd, \$rs, constante	AND inmediato: $d = s \& \text{constante}$
slti 	I	slti \$rd, \$rs, constante	SLT inmediato: Si $s < \text{constante}$, $d = 1$
lw 	I	lw \$rd, desplazamiento(\$rs)	$rd = rs + \text{desplazamiento}$
sw 	I	sw \$rd, desplazamiento(\$rs)	$rs + \text{desplazamiento} = rd$
beq 	I	beq \$rs, \$rt, etiqueta	Branch if Equal: Salta si $s = t$
bne 	I	bne \$rs, \$rt, etiqueta	Branch if Not Equal: Salta si $s \neq t$
bgtz 	I	bgtz \$rs, etiqueta	Branch if Greater Than Zero: Salta si $s > 0$
j 	J	j etiqueta	Salto incondicional a dirección de etiqueta
xor 	R	xor \$rd, \$rs, \$rt	$d = 1$, si los bits de s y t son diferentes
teq 	R	teq \$rs, \$rt	Excepción si el contenido de $s = t$
tge 	R	tge \$rs, \$rt	Excepción si $s \geq t$
sra 	R	sra \$rd, \$rt, shamt	Desplaza bits de t a la derecha shamt veces, manteniendo el bit de signo
srl 	R	srl \$rd, \$rt, shamt	Desplaza los bits de t a la derecha shamt veces y rellena dependiendo del signo

-  Instrucciones Aritmeticas y Logicas
-  Instrucciones con Inmediatos
-  Instrucciones de Memoria (Carga y Almacenamiento)
-  Instrucciones de Control de Flujo (Saltos)
-  Instrucciones de Desplazamiento
-  Instrucciones de Excepción (Trap)

2. Objetivos

2.1 Objetivo general

Explicar el funcionamiento del Datapath e implementar el diseño, tomando en cuenta todos los módulos necesarios para su funcionamiento, tales como, el banco de registros, ALU, multiplexores, unidades de control, desde la investigación, organización, división de tareas, implementación y ejecución, utilizando para esta primera fase las instrucciones de tipo R especificadas en la tabla descrita dentro de la introducción.

2.1 Objetivos específicos

- Investigar de manera profunda cada concepto para su entendimiento y aplicación.
- Desarrollar el código de prueba en ensamblador para instrucciones tipo R.
- Hacer las modificaciones necesarias a los módulos anteriormente diseñados para ejecutar todas las instrucciones tipo R de la tabla.
- Hacer el script de Python para convertir el archivo de código ensamblador a binario.
- Realizar los Test Bench necesarios para comprobar el funcionamiento de la práctica.

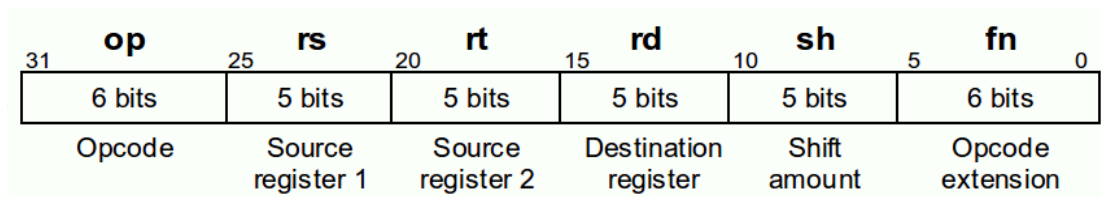
3. Marco Teórico

3.1 Instrucción Tipo R

En la arquitectura MIPS, las instrucciones tipo R (Register-type) se utilizan para realizar operaciones entre registros. Este tipo de instrucción se caracteriza por operar exclusivamente con datos almacenados en registros internos del procesador, sin acceso directo a la memoria.

El formato (máquina) de instrucción tipo R se compone de los siguientes campos:

- **opcode:** código de operación (generalmente 0 en tipo R)
- **rs:** primer registro fuente
- **rt:** segundo registro fuente
- **rd:** registro destino
- **shamt:** desplazamiento (shift amount)
- **funct:** especifica la operación exacta



Representación de la separación de bits para cada campo.

Como se observa en la imagen, de los 32 bits provenientes de la instrucción el campo function y el opcode necesitan de 6 bits para llevar a cabo la operación, y los campos restantes constan de 5 bits.

De acuerdo a esta información, la separación de campos en verilog para instrucciones del tipo R en formato MIPS, según los bits especificados para cada apartado, quedaría algo así:

```
assign opcode = instruccion[31:26];
assign rs     = instruccion[25:21];
assign rt     = instruccion[20:16];
assign rd     = instruccion[15:11];
assign shamt  = instruccion[10:6];
assign funct  = instruccion[5:0];
```

Opcode y **funct** con 6 bits cada uno, **rs**, **rt**, **rd** y **shamt** con 5 bits, dando el total de 32 bits para la instrucción.

Sintaxis para Ensamblador

La sintaxis básica para instrucciones tipo R (Registro-Registro) en lenguaje ensamblador, es la siguiente:

opcode registro_destino , registro_fuente1 , registro_fuente2

Ejemplo:

add \$7 , \$4 , \$2

Ejemplo de la sintaxis para una suma en código ensamblador, se usa el operador add. Al número en la dirección de memoria 4 se le suma el número en la dirección de memoria 2, el resultado se escribe en la dirección de memoria 7.

Elementos de la Sintaxis

- **Mnemónico (Opcode):** Nombre de la instrucción (ej. ADD, SUB, AND, OR, SLT).
- **Registro Destino (rd):** El primer operando tras el mnemónico, donde se almacenará el resultado.
- **Registro Fuente 1 (rs):** Segundo operando, primer valor de entrada a la ALU.
- **Registro Fuente 2 (rt):** Tercer operando, segundo valor de entrada a la ALU.

Operaciones implementables

Existe una gran cantidad de operaciones aritmético-lógicas a realizar con las instrucciones tipo R, tales como add(suma), sub(resta), and, or. Dentro de estas una de las operaciones más relevantes es **SLT** (Set on Less Than). Esta instrucción compara dos registros y establece el valor del registro destino dependiendo del resultado de la comparación:

- Si **rs < rt**, entonces **rd = 1**
- En caso contrario, **rd = 0**

La instrucción SLT es fundamental para la implementación de estructuras de control en bajo nivel, como condicionales (if) y ciclos (while), ya que permite realizar comparaciones entre valores.

Internamente la ALU no puede realizar la comparativa entre un valor y otro e indicar si es menor o mayor directamente, es por eso que, esta operación suele implementarse mediante una resta (**rs - rt**) y la evaluación del bit de signo del resultado.

3.2 Operador Ternario

La operación ternaria es común en lenguajes de programación de alto nivel como C y Verilog. Se trata de una forma compacta de expresar una estructura condicional.

La sintaxis de este operador en C sería la siguiente:

condición ? valor_si_verdadero : valor_si_falso;

Esta operación evalúa una condición lógica. Si la condición es verdadera, retorna el primer valor, si es falsa, retorna el segundo.

En el contexto de diseño digital (por ejemplo, en Verilog), la operación ternaria se utiliza frecuentemente para describir lógica combinacional. Por ejemplo:

assign salida = (a < b) ? 1 : 0;

Aquí se observa una equivalencia directa con la instrucción SLT, ya que se está evaluando una condición de comparación y asignando un valor binario como resultado. Es útil, ya que simplifica el código y puede representar directamente multiplexores en hardware, lo que facilita la síntesis.

3.3 Desglose de Instrucciones Elegidas

Las instrucciones xor, sra y srl, son lógicas y de desplazamiento, estas instrucciones operan a nivel de bits o desplazan la posición de los mismos dentro de un registro.

- **XOR** (lógica): Operación or exclusiva bit a bit entre los valores de 2 registros y almacena el resultado en un tercer registro.
- **SRL** (lógica): Siempre rellena con 0.
- **SRA** (aritmética): Rellena con el valor del bit de signo (si el número era negativo, rellena con 1: si era positivo, con 0) para preservar el valor matemático.

Las instrucciones como teq y tge, de tipo R, comparan dos registros. Si la condición se cumple, el procesador genera una excepción (interrupción de software), lo que suele detener el programa o saltar a un manejador de errores.

- **TEQ** (excepción): Compara dos registros y genera una excepción (trampa) si son iguales, se utiliza para gestionar errores de software, como la división por cero, terminando el programa y transfiriendo el control al gestor de excepciones.
- **TGE** (excepción): Genera una trampa si $rs \geq rt$. Se usa comúnmente para verificar límites, depuración, validación de índices de arreglos.

3.4 Fases del Proceso de Compilación

Este es el proceso que transforma un programa escrito en un lenguaje de programación de alto nivel (código fuente) a un lenguaje de bajo nivel (código objeto o binario). A diferencia de un intérprete (que traduce línea por línea mientras el programa corre), el compilador traduce todo el programa antes de que se ejecute.

El transcurso desde que presionamos “compilar” hasta que obtienes un archivo .exe o un binario para MIPS, como será en esta práctica, se divide en etapas o fases:

- ❖ **Análisis Léxico (Scanner)**

El compilador lee el código fuente carácter por carácter y los agrupa en unidades con significado llamadas *tokens*. Por ejemplo: Si escribes `suma = 5 + 10`, el analizador identifica: un identificador (suma), un operador de asignación (=), un número (5), etc.

- ❖ **Análisis Sintáctico (Parser)**

Aquí se revisa que los *tokens* sigan las reglas gramaticales del lenguaje. Se crea una estructura jerárquica llamada Árbol de Sintaxis Abstracta (AST). Si te falta un punto y coma o un paréntesis, aquí es donde el compilador te lanza el error.

- ❖ **Análisis Semántico**

El compilador verifica que el código tenga sentido, por ejemplo, al sumar una palabra con un número, o usar una variable no declarada, si es así, el proceso se detiene.

- ❖ **Generación de Código Intermedio**

El compilador crea una versión del código que es independiente de la máquina. Es una representación simplificada que facilita las optimizaciones antes de pasar al lenguaje específico de un procesador.

- ❖ **Optimización de Código**

Esta es la fase "inteligente". El compilador intenta que el programa sea más rápido o ocupe menos memoria. Por ejemplo, si tienes una operación como `x = 2 * 4`, el optimizador la cambia directamente por `x = 8` para ahorrarle trabajo al procesador.

- ❖ **Generación de Código de Salida**

El compilador traduce el código optimizado al lenguaje específico de la arquitectura destino (como las instrucciones de 32 bits de MIPS o de x86).

¿Qué pasa cuando se completaron las fases?

El compilador, no es el proceso completo. Después de compilar, se obtiene un “archivo objeto” (.o o .obj), pero este aun no funciona solo.

El **Linker** o enlazador toma el archivo objeto y lo une con las librerías del sistema (por ejemplo, las que permiten imprimir en pantalla o leer el teclado) para generar el ejecutable final.

Entonces se inicia por un **análisis**, entender que escribió el programador y detectar errores de escritura, pasa por una **síntesis**, construir el programa en lenguaje máquina y optimizarlo, y finalmente llegamos al **enlazado**, unir todas las piezas y librerías en un solo archivo funcional.

3.5 Algoritmos Implementables con la ISA

De acuerdo al set de instrucciones definido para este proyecto, se pueden llegar a construir algoritmos con cierta complejidad lógica y aritmética, combinando instrucciones de registro, inmediatas y saltos.

Estructuras de control de flujo (lógica condicional)

Con instrucciones de salto y comparación, podemos recrear cualquier lógica de decisión:

- **Sentencias if-else:** Utilizando beq y bne para saltar a diferentes bloques de código según si dos registros son iguales o no.
- **Bucles (for, while):** Combinando saltos condicionales (bgtz, beq) con saltos incondicionales (j) al final del bloque para repetir una secuencia.
- **Validaciones críticas:** Usando las instrucciones de excepción teq y tge para detener el programa si ocurre un error lógico (como un índice fuera de rango o una condición prohibida).

Manipulación de Estructuras de Datos

Con las instrucciones de carga y almacenamiento, puedes gestionar datos en la memoria RAM:

- **Recorrido de Arreglos (Arrays):** Utilizando lw (cargar) y sw (guardar) junto con addi para incrementar el puntero o índice de memoria.
- **Copia de datos:** Algoritmos para mover bloques de información de una dirección de memoria a otra.

Aritmética y Procesamiento de Bits

- **Cálculos Matemáticos:** Implementación de fórmulas complejas usando add, sub y addi.
- **Multiplicación y División por potencias de 2:** Usando srl y sra para desplazar bits, lo cual es mucho más rápido que realizar operaciones matemáticas completas.
- **Máscaras de Bits:** Con and, or, xor y andi, puedes extraer, encender o invertir bits específicos dentro de una palabra de datos (útil para protocolos de comunicación o control de hardware).

Algoritmos de Ordenamiento y Búsqueda

Es posible implementar algoritmos clásicos de ciencias de la computación:

- **Búsqueda Lineal:** Recorriendo la lista con lw y comparando con beq.
- **Ordenamiento** (como el método de la burbuja): Utilizando slt o slti para comparar valores y sw para intercambiar sus posiciones en memoria si uno es menor que otro.

3.6 Búsqueda Binaria (Algoritmo) Aún no se implementa en esta fase

La búsqueda binaria es uno de los algoritmos de búsqueda más eficientes en informática, el método que utiliza es sencillo, ya que, para encontrar algo, no necesitas mirar cada elemento, solo necesitas saber hacia qué dirección moverte.

El algoritmo utiliza una estrategia de división sucesiva. En lugar de revisar uno por uno (como la búsqueda secuencial), este método divide el espacio de búsqueda a la mitad en cada paso. Para que la búsqueda binaria funcione, la colección de datos (arreglo o lista) debe estar ordenada (ya sea de forma ascendente o descendente). Si los datos están desordenados, el algoritmo fallará, ya que no podrá descartar mitades con seguridad.

Funcionamiento

Imagina que buscas un número X en un arreglo:

- **Definir límites:** Se establecen dos punteros: *Inicio* (posición 0) y *Fin* (última posición).
- **Calcular el punto medio:** Se halla el índice central usando la fórmula:

$$\text{Medio} = \frac{\text{Inicio} + \text{Fin}}{2}$$

- **Comparar:**

- Si el elemento en *Medio* es igual a *X*: La búsqueda termina.
- Si el elemento en *Medio* es mayor que *X*: El número debe estar en la mitad izquierda. Entonces, movemos el puntero *Fin* a *Medio* - 1.
- Si el elemento en *Medio* es menor que *X*: El número debe estar en la mitad derecha. Entonces, movemos el puntero *Inicio* a *Medio* + 1.
- **Repetir:** El proceso vuelve al paso 2 con los nuevos límites hasta encontrar el valor o hasta que Inicio supere a Fin (lo que significa que el valor no existe en la lista).

Lo que hace especial a este algoritmo es su velocidad. Mientras que una búsqueda lineal tarda un tiempo proporcional al número de elementos (n), la búsqueda binaria reduce el problema exponencialmente.

Ventajas	Desventajas
Búsqueda rápida en listas grandes.	Requiere datos pre-ordenados.
Consumo de recursos mínimo.	Menos eficiente en listas pequeñas.
Fácil de implementar recursivamente.	Insertar datos obliga a reordenar.

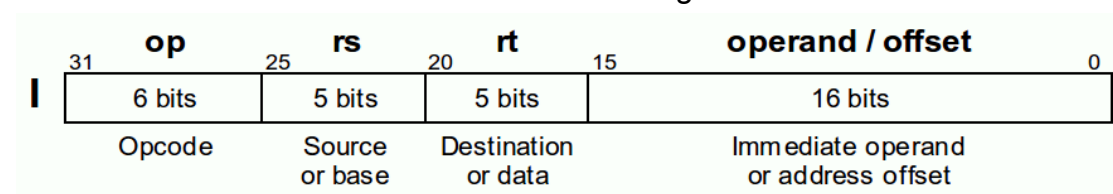
3.7 Instrucciones Tipo I:

Las instrucciones Tipo I (Immediate-type) son fundamentales en la arquitectura MIPS para realizar operaciones que involucran operandos constantes (inmediatos) y para gestionar la transferencia de datos entre los registros del procesador y la memoria principal (RAM) . A diferencia de las instrucciones puramente aritméticas que operan sobre datos preexistentes en registros, el formato Tipo I dota al procesador de la capacidad de introducir nuevos valores al sistema y de acceder a estructuras de datos masivas guardadas en memoria.

El formato Tipo I mantiene la longitud estándar de 32 bits, pero redistribuye sus campos para albergar un número constante. Los campos se dividen de la siguiente manera:

- Opcode (Código de Operación): 6 bits que definen la instrucción exacta (ej. lw, sw, addi).
- rs (Register Source): 5 bits que indican el primer registro fuente. En operaciones de memoria, este registro contiene la "dirección base".
- rt (Register Target): 5 bits que indican el registro destino (donde se guardará el resultado) o el registro fuente secundario (cuyo valor se guardará en memoria, como en el caso de sw) .
- Inmediato (Constante / Offset) [15:0]: 16 bits que representan un valor numérico directo o un desplazamiento (offset)

La distribución de los bits se estructura de la siguiente manera:



Diferencias Principales con las Instrucciones Tipo R

La transición del formato Tipo R al Tipo I implica modificaciones sustanciales en la decodificación lógica :

1. Ausencia del campo funct: Las instrucciones Tipo I carecen de los 6 bits finales de función. El opcode principal es el único responsable de indicarle a la Unidad de Control qué operación matemática debe forzar en la ALU.
2. Destino del Registro: En el formato Tipo R, el resultado siempre se almacena en el registro rd (bits 15:11). En el formato Tipo I, al no existir el campo rd, el registro de destino pasa a ser rt (bits 20:16). Esto obliga al hardware a implementar un multiplexor (RegDst) para decidir qué bits definen la dirección de escritura.
3. Operando Inmediato: Se sacrifica el tercer registro para permitir la incrustación de una constante de 16 bits.

Para el correcto funcionamiento de estas instrucciones, necesitamos agregar un pequeño módulo llamado "Sign extend" o extensor de signo, el cual profundizamos más en un momento.

Extensión de Signo (Sign Extend)

La Unidad Aritmético Lógica (ALU) del procesador MIPS está diseñada para operar exclusivamente con datos de 32 bits de longitud. Dado que el campo inmediato de la instrucción Tipo I aporta únicamente 16 bits, el hardware se enfrenta a una incompatibilidad de tamaño.

Para resolver esto sin alterar el valor matemático (especialmente si el número es negativo en complemento a dos), se implementa un módulo combinacional denominado Extensor de Signo (Sign Extend). Este módulo toma los 16 bits del inmediato y replica el bit más significativo (el bit 15, correspondiente al signo) 16 veces hacia la izquierda para completar una palabra de 32 bits .

En Verilog, esta operación se logra de forma eficiente mediante la concatenación:

```
assign out = {{16{in[15]}}, in};
```

Instrucciones de Memoria: Load Word (lw) y Store Word (sw)

Las instrucciones lw y sw utilizan el formato Tipo I para implementar el concepto de direccionamiento base más desplazamiento. Lw significa "load word", y en este contexto lo que carga es un dato, de nuestra memoria al banco de registros, su propósito general es obtener variables para hacer operaciones matemáticas o lógicas. Sw significa store word y hace lo contrario a lw, guarda el registro que

tenemos en nuestra memoria de datos, su propósito general es guardar resultados finales o valores temporales en la memoria.

Sintaxis en Ensamblador de Lw y Sw:

lw \$rt, desplazamiento(\$rs) : Carga una palabra (32 bits) de la memoria hacia el registro rt.

sw \$rt, desplazamiento(\$rs) : Guarda la palabra del registro rt en la memoria.

Instrucción Beq:

La instrucción beq (Branch if Equal) en MIPS es de tipo I (Inmediato) y se utiliza para tomar decisiones. Compara el contenido de dos registros; si son iguales, realiza un salto a una etiqueta; si no, continúa con la siguiente instrucción

Funcionamiento:

1. El procesador calcula la dirección de destino de la siguiente forma: Se incrementa el Contador de Programa (PC) para que apunte a la instrucción siguiente
2. El campo "Inmediato" (offset) se extiende de signo a 32 bits y se multiplica por 4 (ya que en MIPS las instrucciones están alineadas a palabras de 4 bytes).
3. Si los registros son iguales, la nueva dirección de ejecución será:
$$PC_nueva = (PC + 4) + (Inmediato \times 4)$$

Si no son iguales, simplemente se ejecuta $PC+4$

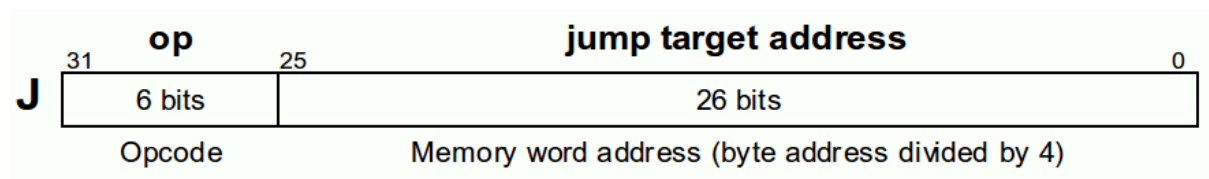
3.7 Instrucciones Tipo J

Las instrucciones Tipo J (Jump) se utilizan en la arquitectura MIPS para realizar saltos incondicionales hacia una dirección específica de la memoria de instrucciones. A diferencia de las instrucciones de salto condicional Tipo I (como beq o bne), que dependen del resultado de una comparación en la ALU para decidir si saltan o no, las instrucciones Tipo J rompen la secuencialidad del programa de forma obligatoria y directa cada vez que son ejecutadas. Su propósito principal es permitir la transferencia del control del flujo del programa hacia subrutinas, funciones o etiquetas lejanas que se encuentran fuera del rango de alcance de los saltos relativos de Tipo I.

Formato de Máquina (Codificación de Bits)

El formato de una instrucción Tipo J consta de 32 bits en total. Para maximizar la cantidad de memoria a la que se puede saltar, este formato prescinde por completo del uso de registros y divide sus campos en únicamente dos apartados:

- Opcode (Código de Operación): Ocupa 6 bits y define la operación específica de salto. Para la instrucción estándar j, el código binario asignado es 000010.
- Target Address (Dirección Objetivo) [25:0]: Ocupa los 26 bits restantes de la instrucción. Contiene la dirección codificada a la que el procesador debe desviar el Program Counter (PC)



Para saltar a una dirección de 32 bits, el procesador reconstruye la posición destino de la siguiente manera:

1. Toma los 26 bits del campo de dirección y los desplaza 2 posiciones a la izquierda (lo que equivale a multiplicar por 4).
2. Toma los 4 bits más significativos del registro Contador de Programa (PC +4).
3. Concatena ambos para formar la dirección final de 32 bits a la que saltará el procesador.

Diferencias Principales con las Instrucciones Tipo R

Es fundamental contrastar el formato Tipo J con el formato Tipo R trabajado en la primera fase para comprender la optimización del hardware:

1. Uso de Registros: Las instrucciones Tipo R operan exclusivamente con datos almacenados en los registros internos (rs, rt, rd). Las instrucciones Tipo J no interactúan con el Banco de Registros; operan directamente sobre el Program Counter (PC) utilizando un valor inmediato .
2. Especificación de la Operación: En el formato Tipo R, el opcode general suele ser 000000, y la operación exacta de la ALU se determina mediante los 6 bits del campo funcional (funct) al final de la instrucción. En el formato Tipo J, el opcode de 6 bits define por sí mismo y de manera absoluta la acción de salto, eliminando la necesidad de un campo funct.

3. Uso de los Bits: Tipo R divide sus 32 bits en 6 campos pequeños para direccionar múltiples registros y desfases (shamt). Tipo J fusiona 26 bits en un solo bloque para disponer del mayor espacio posible para una dirección de memoria.

4. Desarrollo

4.1 Pseudocódigo Algoritmo Ensamblador

Búsqueda Binaria

(Inicialización de variables)

izq <- 0

der <- 0

Mientras NO (der < izq) Hacer:

suma_indices <- izq + der

medio <-

Desplazar_Derecha_Bits (Suma_Indices 1)

Offset_temp <- medio + medio

Offset_final <- Offset_temp + Offset_temp

Direccion_actual <- DireccionBase_A + Offset_final

Si valor_leido == x Entonces:

Retornar medio

Si no, Si valor_leido < x Entonces:

izq <- medio + 1

Si no

der <- medio - 1

Fin Si

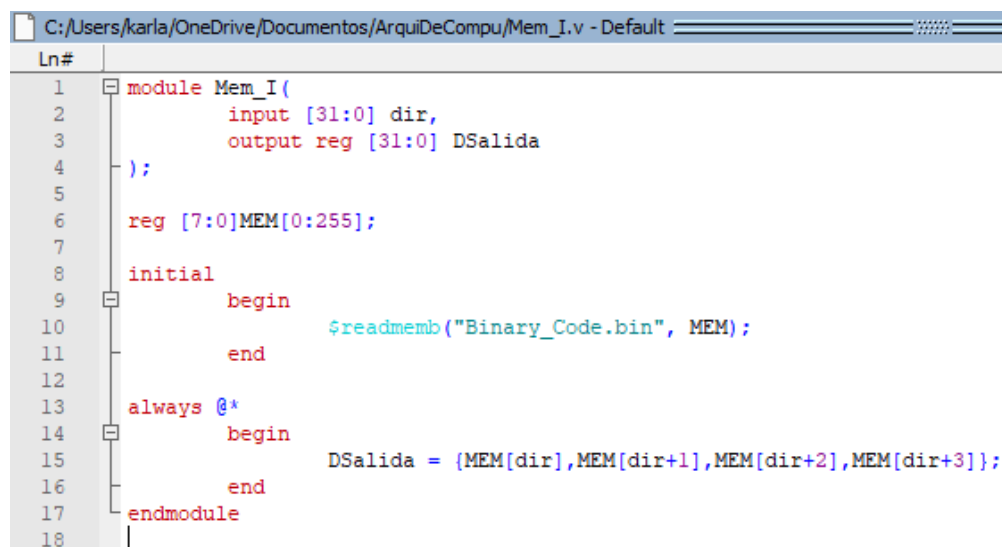
Fin Mientras

Retornar -1

4.2 Módulos Modificados (lectura archivo .bin)

Módulo Memoria de Instrucciones

```
module Mem_I(  
    input [31:0] dir,  
    output reg [31:0] DSalida  
);  
  
reg [7:0]MEM[0:255];  
  
initial  
    begin  
        $readmemb("Binary_Code.bin", MEM);  
    end  
  
always @*  
    begin  
        DSalida = {MEM[dir],MEM[dir+1],MEM[dir+2],MEM[dir+3]};  
    end  
endmodule
```



Módulo Mem_I en ModelSim.

Módulo Banco de Registros

```
module BancoReg (  
    input clk,  
    input RegWrite,  
    input [4:0] rs, rt, rd,  
    input [31:0] writeData,  
    output [31:0] readData1, readData2  
);  
  
reg [31:0] regs [0:31];
```

```

//inicializar para evitar el error
integer i;
initial begin
    for (i = 0; i < 32; i = i + 1) begin
        regs[i] = 32'b0;
    end
end

assign readData1 = regs[rs];
assign readData2 = regs[rt];

always @(posedge clk) begin
    if (RegWrite && rd != 0)
        regs[rd] <= writeData;
end

endmodule

```

```

C:/Users/karla/OneDrive/Documentos/ArquiDeCompu/Bank_R.v (TB_DPTR/tm/dp)
Ln#
1 module BancoReg (
2     input clk,
3     input RegWrite,
4     input [4:0] rs, rt, rd,
5     input [31:0] writeData,
6     output [31:0] readData1, readData2
7 );
8
9 reg [31:0] regs [0:31];
10
11 //inicializar para evitar el error
12 integer i;
13 initial begin
14     for (i = 0; i < 32; i = i + 1) begin
15         regs[i] = 32'b0;
16     end
17 end
18
19 assign readData1 = regs[rs];
20 assign readData2 = regs[rt];
21
22 always @(posedge clk) begin
23     if (RegWrite && rd != 0)
24         regs[rd] <= writeData;
25 end
26
27 endmodule

```

Módulo BancoReg en ModelSim.

Módulo Unidad de Control

```

module U_Control(
    input [5:0]op,
    input [5:0]funct,

```

```

        output reg memToReg,
        output reg memToWrite,
        output reg memToRead,
        output reg [2:0]AluOP,
        output reg RegWrite
    );
    always@(*)
    begin
        RegWrite = 1'b0;
        AluOP = 3'b000;
        memToRead = 1'b0;
        memToWrite = 1'b0;
        memToReg = 1'b0;

    case (op)
        6'b000000:
            begin
                AluOP = 3'b010;
                if(funcnt == 6'b110100 || funcnt == 6'b110000)//para teq y tge
                    RegWrite = 1'b0;
                else
                    RegWrite = 1'b1;
            end
    endcase
    end
endmodule

```

```

C:/Users/karla/OneDrive/Documentos/ArquiDeCompu/U_Control.v (/TB_DPTR/tm/dp/UC) - Default
Ln#
1  module U_Control(
2      input [5:0]op,
3      input [5:0]funct,
4      output reg memToReg,
5      output reg memToWrite,
6      output reg memToRead,
7      output reg [2:0]AluOP,
8      output reg RegWrite
9  );
10 always@(*)
11 begin
12     RegWrite = 1'b0;
13     AluOP = 3'b000;
14     memToRead = 1'b0;
15     memToWrite = 1'b0;
16     memToReg = 1'b0;
17
18     case (op)
19         6'b000000:
20         begin
21             AluOP = 3'b010;
22             if(funct == 6'b110100 || funct == 6'b110000)//para teq y tge
23                 RegWrite = 1'b0;
24             else
25                 RegWrite = 1'b1;
26         end
27     endcase
28 end
29 endmodule

```

Módulo U_Control en ModelSim.

Módulo ALU Control

```

module ALUControl (
    input [5:0] fnc,
    input [2:0] ALUOp,
    output reg [3:0] Sel
);
always @(*) begin
    case (ALUOp)
        3'b010: begin
            case (fnc)
                6'b100000: Sel = 4'b0010; // add
                6'b100010: Sel = 4'b0110; // sub
                6'b100100: Sel = 4'b0000; // and
                6'b100101: Sel = 4'b0001; // or
                6'b101010: Sel = 4'b0111; // sl
                6'b100110: Sel = 4'b0011; // xor
                6'b000000: Sel = 4'b1111; // nop
                6'b000011: Sel = 4'b1000; // sra
                6'b000010: Sel = 4'b1001; // srl
                6'b110100: Sel = 4'b1100; // teq
                6'b110000: Sel = 4'b1101; // tge
                default: Sel = 4'b0000;
            endcase
        end
    endcase
end

```

```

        endcase
    end
endcase
end
endmodule

```

```

C:/Users/karla/OneDrive/Documentos/ArquiDeCompu/ALU_Control.v (/TB_DPTR/tm/dp/ALUC) -
Ln#
1  module ALUControl (
2      input [5:0] fnc,
3      input [2:0] ALUOp,
4      output reg [3:0] Sel
5  );
6      always @(*) begin
7          case (ALUOp)
8              3'b010: begin
9                  case (fnc)
10                     6'b100000: Sel = 4'b0010; // add
11                     6'b100010: Sel = 4'b0110; // sub
12                     6'b100100: Sel = 4'b0000; // and
13                     6'b100101: Sel = 4'b0001; // or
14                     6'b101010: Sel = 4'b0111; // sl
15                     6'b100110: Sel = 4'b0011; // srl
16                     6'b000000: Sel = 4'b1111; // nop
17                     6'b000011: Sel = 4'b1000; // sra
18                     6'b000010: Sel = 4'b1001; // srl
19                     6'b110100: Sel = 4'b1100; // teq
20                     6'b110000: Sel = 4'b1101; // tge
21                 default: Sel = 4'b0000;
22             endcase
23         end
24     endcase
25 end
26 endmodule

```

Módulo ALUControl en ModelSim.

Módulo ALU

```

module ALU (
    input [31:0] A, B,
    input [4:0] shamt, //para sra y srl
    input [3:0] ALUCtrl,
    output reg [31:0] Result,
    output reg Exception //senal de teq y tge
);
always @(*) begin
    Result = 0;
    Exception = 0;
    case (ALUCtrl)
        4'b0010: Result = A + B; // add
        4'b0110: Result = A - B; // sub
        4'b0000: Result = A & B; // and
        4'b0001: Result = A | B; //or
        4'b0111: Result = (A < B) ? 1:0; // slt

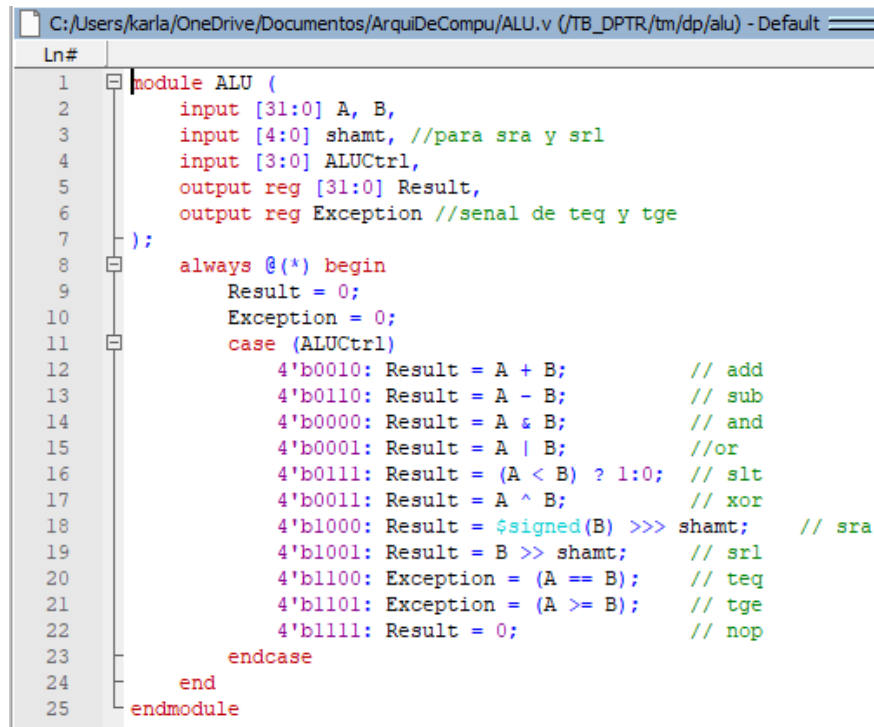
```



```

4'b0011: Result = A ^ B;      // xor
4'b1000: Result = $signed(B) >>> shamt; // sra
4'b1001: Result = B >> shamt; // srl
4'b1100: Exception = (A == B); // teq
4'b1101: Exception = (A >= B); // tge
4'b1111: Result = 0;          // nop
endcase
end
endmodule

```



Módulo ALU en ModelSim.

Módulo DPTR

```

module DPTR (
    input clk,
    input [31:0] instruction,
    output [31:0] result
);
    wire [5:0] op = instruction[31:26];
    wire [4:0] rs = instruction[25:21];
    wire [4:0] rt = instruction[20:16];
    wire [4:0] rd = instruction[15:11];
    wire [4:0] sh = instruction[10:6];
    wire [5:0] funct = instruction[5:0];

    wire memToReg, RegWrite, memToWrite, memToRead;
    wire [2:0] ALUOp;
    wire [3:0] ALUCtrl;
    wire aluException;

```

```

wire [31:0] readData1, readData2, aluResult;

U_Control UC(op, funct, memToReg, memToWrite, memToRead, ALUOp,
RegWrite);
ALUControl ALUC(funct, ALUOp, ALUCtrl);
BancoReg BR(clk, RegWrite, rs, rt, rd, aluResult, readData1, readData2);
ALU alu(readData1, readData2, sh, ALUCtrl, aluResult, aluException);

assign result = aluResult;
endmodule

```

```

C:\Users\karla\OneDrive\Documentos\ArquiDeCompu\DPTR.v (/TB_DPTR/tm/dp) - Default
Ln#
1  module DPTR (
2      input clk,
3      input [31:0] instruction,
4      output [31:0] result
5  );
6      wire [5:0] op = instruction[31:26];
7      wire [4:0] rs = instruction[25:21];
8      wire [4:0] rt = instruction[20:16];
9      wire [4:0] rd = instruction[15:11];
10     wire [4:0] sh = instruction[10:6];
11     wire [5:0] funct = instruction[5:0];
12
13     wire memToReg, RegWrite, memToWrite, memToRead;
14     wire [2:0] ALUOp;
15     wire [3:0] ALUCtrl;
16     wire aluException;
17     wire [31:0] readData1, readData2, aluResult;
18
19     U_Control UC(op, funct, memToReg, memToWrite, memToRead, ALUOp, RegWrite);
20     ALUControl ALUC(funct, ALUOp, ALUCtrl);
21     BancoReg BR(clk, RegWrite, rs, rt, rd, aluResult, readData1, readData2);
22     ALU alu(readData1, readData2, sh, ALUCtrl, aluResult, aluException);
23
24     assign result = aluResult;
25 endmodule

```

Módulo DPTR en ModelSim.

Módulo Memoria de Datos

```

module Mem_D(
    input clk,
    input MemWrite,    //1 para escribir, 0 para leer
    input [31:0] dir,
    input [31:0] DEntrada,    output reg [31:0] DSalida
);

reg [31:0] MEM [0:255];

always @*
begin
    DSalida = MEM[dir];
end

always @(posedge clk)
begin

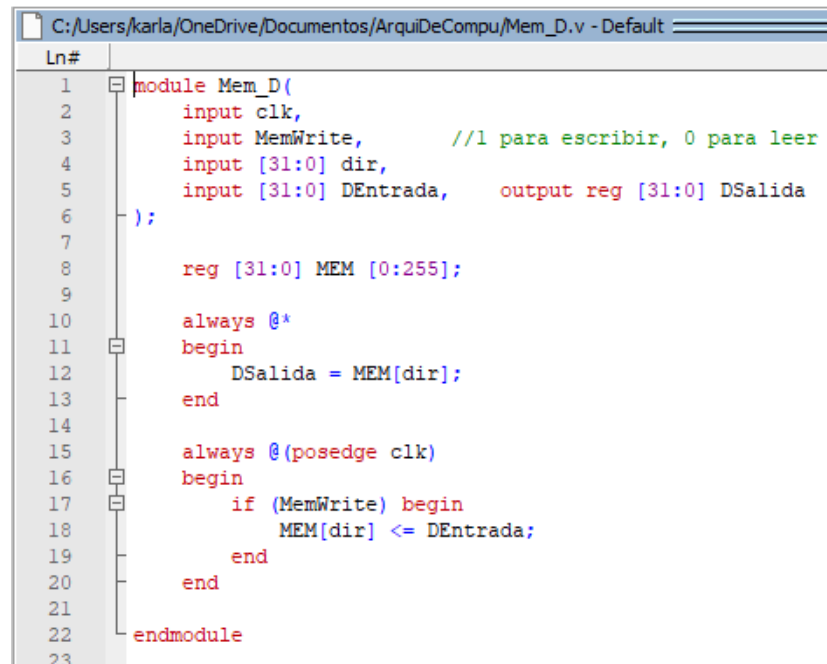
```

```

        if (MemWrite) begin
            MEM[dir] <= DEntrada;
        end
    end

endmodule

```



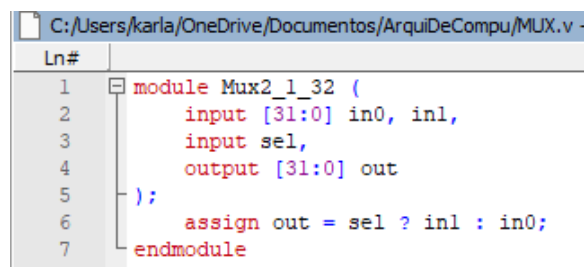
Módulo Mem_D en ModelSim.

Módulo Multiplexor

```

module Mux2_1_32 (
    input [31:0] in0, in1,
    input sel,
    output [31:0] out
);
    assign out = sel ? in1 : in0;
endmodule

```



Módulo Mux2_1_32 en ModelSim.

Módulo Test Bench DPTR

```

module TB_DPTR;
    reg clk;
    reg reset;
    wire [31:0] result;

    MIPS_TM tm(clk, reset, result);

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        reset = 1;
        #1;

        tm.dp.BR.regs[8] = 32'd20;
        tm.dp.BR.regs[9] = 32'd40;
        tm.dp.BR.regs[10] = 32'd60;
        tm.dp.BR.regs[11] = 32'd80;
        tm.dp.BR.regs[12] = 32'd5;
        tm.dp.BR.regs[13] = 32'd15;
        tm.dp.BR.regs[17] = 32'd100;
        tm.dp.BR.regs[18] = 32'd50;
        tm.dp.BR.regs[19] = 32'd25;
        tm.dp.BR.regs[20] = 32'd30;
        tm.dp.BR.regs[21] = 32'd10;

        tm.dp.BR.regs[0] = 32'd0;

        #9;
        reset = 0;

        $display("Ejecutando Archivo .bin");

        #150;

        $display("Simulacion finalizada");

        $stop;
    end
endmodule

```

```

C:/Users/karla/OneDrive/Documentos/ArquiDeCompu/TB_DPTR.v (/TB_DPTR) - Default
Ln#
1  module TB_DPTR;
2      reg clk;
3      reg reset;
4      wire [31:0] result;
5
6      MIPS_TM tm(clk, reset, result);
7
8      initial begin
9          clk = 0;
10         forever #5 clk = ~clk;
11     end
12
13     initial begin
14         reset = 1;
15         #1;
16
17         tm.dp.BR.regs[8] = 32'd20;
18         tm.dp.BR.regs[9] = 32'd40;
19         tm.dp.BR.regs[10] = 32'd60;
20         tm.dp.BR.regs[11] = 32'd80;
21         tm.dp.BR.regs[12] = 32'd5;
22         tm.dp.BR.regs[13] = 32'd15;
23         tm.dp.BR.regs[17] = 32'd100;
24         tm.dp.BR.regs[18] = 32'd50;
25         tm.dp.BR.regs[19] = 32'd25;
26         tm.dp.BR.regs[20] = 32'd30;
27         tm.dp.BR.regs[21] = 32'd10;
28
29         tm.dp.BR.regs[0] = 32'd0;
30
31         #9;
32         reset = 0;
33
34         $display("Ejecutando Archivo .bin");
35
36         #150;
37
38         $display("Simulacion finalizada");
39
40         $stop;
41     end
42 endmodule

```

Módulo TB_DPTR en ModelSim.

Módulo Program Counter

```

module PC (
    input clk,
    input reset,
    output reg [31:0] pc_out
);
    always @(posedge clk or posedge reset) begin
        if (reset)
            pc_out <= 32'b0;
        else
            pc_out <= pc_out + 1;
        end
    endmodule

```

```

1  module PC (
2      input clk,
3      input reset,
4      output reg [31:0] pc_out
5  );
6      always @(posedge clk or posedge reset) begin
7          if (reset)
8              pc_out <= 32'b0;
9          else
10             pc_out <= pc_out + 1;
11         end
12     endmodule
13

```

Módulo PC en ModelSim.

Top Module MIPS

```

module MIPS_TM (
    input clk,
    input reset,
    output [31:0] out_result
);
    wire [31:0] pc_to_mem;
    wire [31:0] instr_to_dp;

    PC pc1(clk, reset, pc_to_mem);
    Mem_I memi(pc_to_mem, instr_to_dp);
    DPTR dp(clk, instr_to_dp, out_result);
endmodule

```

```

1  module MIPS_TM (
2      input clk,
3      input reset,
4      output [31:0] out_result
5  );
6      wire [31:0] pc_to_mem;
7      wire [31:0] instr_to_dp;
8
9      PC pc1(clk, reset, pc_to_mem);
10     Mem_I memi(pc_to_mem, instr_to_dp);
11     DPTR dp(clk, instr_to_dp, out_result);
12 endmodule
13

```

Módulo MIPS_TM en ModelSim.

4.3 Ensamblador, script Python y .bin

Ensamblador (.asm)

PRUEBA DE INSTRUCCIONES TIPO R

Aritméticas y Lógicas (rd, rs, rt)

add \$t0, \$s1, \$s2

sub \$t1, \$t0, \$s3

and \$t2, \$s4, \$s5

or \$t3, \$t1, \$t2

xor \$t4, \$t2, \$t3

slt \$t5, \$s1, \$s2

Desplazamientos (rd, rt, shamt)

rs debe quedar en 00000 internamente

srl \$s6, \$t4, 2

sra \$s7, \$t5, 4

Trampas (rs, rt)

rd y shamt quedan en 00000

teq \$s1, \$s2

tge \$t0, \$t1

Operación Nula

nop

Script de Python (Conversor)

import tkinter as tk

from tkinter import filedialog, messagebox

Tablas MIPS

REG = {

"\$zero": "00000", "\$at": "00001", "\$v0": "00010", "\$v1": "00011",
"\$a0": "00100", "\$a1": "00101", "\$a2": "00110", "\$a3": "00111",
"\$t0": "01000", "\$t1": "01001", "\$t2": "01010", "\$t3": "01011",
"\$t4": "01100", "\$t5": "01101", "\$t6": "01110", "\$t7": "01111",
"\$s0": "10000", "\$s1": "10001", "\$s2": "10010", "\$s3": "10011",
"\$s4": "10100", "\$s5": "10101", "\$s6": "10110", "\$s7": "10111",
"\$t8": "11000", "\$t9": "11001"

}

FUNCT = {

"add": "100000", "sub": "100010", "and": "100100", "or": "100101",
"xor": "100110", "slt": "101010", "srl": "000010", "sra": "000011",
"tge": "110000", "teq": "110100"

}

```

def to_bin(val, bits):
    """Convierte un número a binario con n bits."""
    return format(int(val), f'0{bits}b')

# --- LÓGICA DE CONVERSIÓN ---
def ensamblar_linea(linea):
    linea = linea.split('#')[0].replace(',', ' ').strip().lower()
    if not linea: return None

    partes = linea.split()
    inst = partes[0]

    if inst == "nop":
        return "0" * 32

    # para tipo r se mantiene en 0
    opcode = "000000"

    try:
        if inst in ["add", "sub", "and", "or", "xor", "slt"]:
            rd, rs, rt = partes[1], partes[2], partes[3]
            return opcode + REG[rs] + REG[rt] + REG[rd] + "00000" + FUNCT[inst]

        elif inst in ["srl", "sra"]:
            rd, rt, shamt = partes[1], partes[2], partes[3]
            return opcode + "00000" + REG[rt] + REG[rd] + to_bin(shamt, 5) + FUNCT[inst]

        elif inst in ["tge", "teq"]:
            rs, rt = partes[1], partes[2]
            return opcode + REG[rs] + REG[rt] + "00000" + "00000" + FUNCT[inst]

    except Exception as e:
        return f"ERROR en línea: {linea}"
    return None

# funciones para la interfaz, subida de archivo
def seleccionar_archivo():
    ruta = filedialog.askopenfilename(filetypes=[("MIPS Assembly", "*.asm"), ("Text files", "*.txt")])
    if ruta:
        entrada_ruta.delete(0, tk.END)
        entrada_ruta.insert(0, ruta)

def procesar_archivo():
    ruta_origen = entrada_ruta.get()
    if not ruta_origen:
        messagebox.showwarning("Atención", "Primero carga un archivo .asm")
        return

    try:
        with open(ruta_origen, 'r') as f:

```



```

        lineas = f.readlines()

    resultado = []
    for l in lineas:
        binario = ensamblar_linea(l)
        if binario:
            resultado.append(binario)

    ruta_destino = filedialog.asksaveasfilename(defaultextension=".bin", filetypes=[("Binary
file", "*.bin"), ("Text file", "*.txt")])
    if ruta_destino:
        with open(ruta_destino, 'w') as f:
            f.write("\n".join(resultado))
        messagebox.showinfo("Éxito", "Archivo binario generado correctamente.")

except Exception as e:
    messagebox.showerror("Error", f"No se pudo procesar: {e}")

# disenio ventana
root = tk.Tk()
root.title("MIPS Ensamblador - Solo Tipo R")
root.geometry("500x250")

tk.Label(root, text="Conversor ASM a Binario", font=("Arial", 14, "bold")).pack(pady=10)

# Fila de carga
frame_carga = tk.Frame(root)
frame_carga.pack(pady=10)

btn_cargar = tk.Button(frame_carga, text="Cargar archivo .asm",
command=seleccionar_archivo)
btn_cargar.pack(side=tk.LEFT, padx=5)

entrada_ruta = tk.Entry(frame_carga, width=40)
entrada_ruta.pack(side=tk.LEFT, padx=5)

# Boton convertir
btn_convertir = tk.Button(root, text="CONVERTIR A BINARIO", bg="#4CAF50", fg="white",
font=("Arial", 10, "bold"), command=procesar_archivo)
btn_convertir.pack(pady=20, ipadx=10, ipady=5)

root.mainloop()

```

Archivo en Binario (.bin)

```

00000010
00110010
01000000
00100000
00000001
00010011
01001000

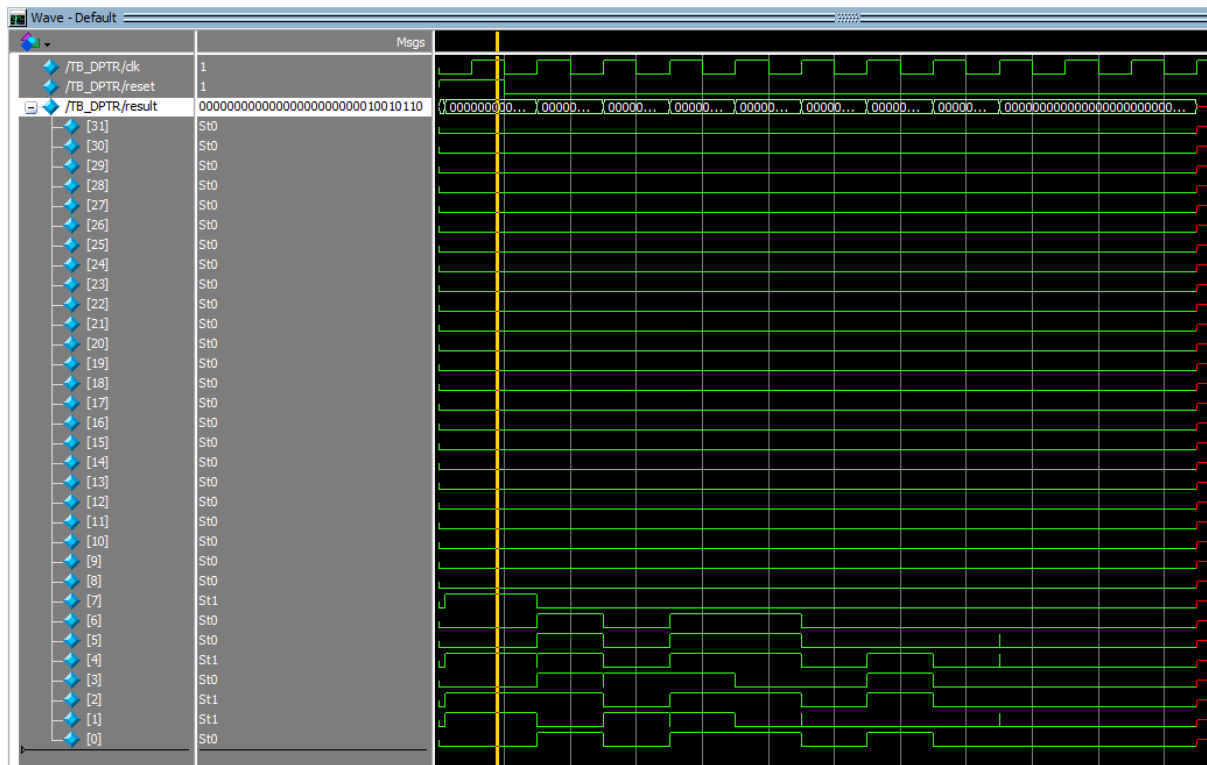
```

```
00100010
00000010
10010101
01010000
00100100
00000001
00101010
01011000
00100101
00000001
01001011
01100000
00100110
00000010
00110010
01101000
00101010
00000000
00001100
10110000
10000010
00000000
00001101
10111001
00000011
00000010
00110010
00000000
00110100
00000001
00001001
00000000
00110000
00000000
00000000
00000000
00000000
```

4.4 Resultados Simulación

Al correr la simulación, nuestro programa lee las instrucciones descritas en ensamblador (archivo .asm), las cuales fueron convertidas a binario mediante el script de python, devolviendonos un archivo .bin (se manda a llamar en el módulo Mem_I con readmemb) con las instrucciones en ceros y unos.

Luego con el módulo del test bench para el DPTR se le da valor a los registros utilizados en las instrucciones tipo R del ensamblador.



Estas son las señales obtenidas en la ventana Wave para el TB.

Memory Data - /TB_DPTR/tm/dp/BR/regs - Default		
0	0	0
2	0	0
4	0	0
6	0	0
8	150	125
10	10	127
12	117	0
14	0	0
16	0	100
18	50	25
20	30	10
22	29	0
24	0	0
26	0	0
28	0	0
30	0	0

Aquí puede observarse el memory list con los valores de cada registro después de correr la simulación, los valores corresponden a los resultados de cada operación.

5. Conclusión

Al final de este proyecto, logramos que el procesador MIPS ejecutara correctamente las instrucciones tipo R desde un archivo binario. Fue una experiencia clave para entender que el hardware no solo requiere lógica, sino una coordinación perfecta entre el reloj, el PC y la memoria.

Aprendimos que pasar del ensamblador al binario mediante Python facilita mucho el trabajo, pero que el verdadero reto está en la simulación; ahí descubrimos que es vital inicializar cada registro y limpiar "la basura" de los datos para evitar errores en las señales. En conclusión, cumplimos el objetivo de integrar todos los módulos y dejamos una base sólida para implementar instrucciones de salto y memoria más adelante.

Comprendimos cómo funcionan las instrucciones tipo I que son las inmediatas y las instrucciones tipo J que son los saltos, para ello tuvimos que implementar un módulo que extendiera el signo donde utilizamos una sintaxis específica en verilog que replica 16 veces lo que hay en cierta posición del arreglo y lo pega en la instrucción.

Utilizamos la memoria de datos para pasar datos de ella misma al banco de registros con la instrucción lw load word y del banco de registros a guardarlo a la memoria con la función sw store word, con esto nuestro procesador ya está más completo y ahora podemos cargarle algoritmos un poco más complejos.

6. Bibliografía

- Wikipedia. (2025). MIPS architecture. Wikipedia. Recuperado de https://en.wikipedia.org/wiki/MIPS_architecture
- GeeksforGeeks. (2024). Instruction format in MIPS. Recuperado de <https://www.geeksforgeeks.org/instruction-format-in-mips/>
- GeeksforGeeks. (2024). Conditional or ternary operator in programming. Recuperado de <https://www.geeksforgeeks.org/conditional-or-ternary-operator-in-programming/>
- Patterson, D. A., & Hennessy, J. L. (2017). Computer organization and design MIPS edition: The hardware/software interface (5th ed.). Morgan Kaufmann.
- Mano, M. M., & Ciletti, M. D. (2017). Digital design (6th ed.). Pearson.
- TutorialsPoint. (2024). Compiler design phases. Recuperado de https://www.tutorialspoint.com/compiler_design/compiler_design_phases.htm
- Nandland. (2023). Verilog conditional operator. Recuperado de <https://nandland.com/conditional-operator/>
- MIPS Technologies, Inc. (2016). MIPS32 Architecture For Programmers Volume I-A: Introduction to the MIPS32 Architecture. <https://www.mips.com>
- Lenovo. (s. f.). *¿Qué es MIPS?*. Glosario de Lenovo. Recuperado el 1 de mayo de 2026, de <https://www.lenovo.com/es/es/glossary/mips/>
- MIPS Assembly/Instruction Formats - Wikibooks, open books for an open world. (n.d.). https://en.wikibooks.org/wiki/MIPS_Assembly/Instruction_Formats
- De Nihil, V. T. L. E. (2019, September 9). Introducción a la Arquitectura del MIPS. La Cripta Del Hacker. <https://lacriptadelhacker.wordpress.com/2019/09/09/introduccion-a-la-arquitectura-de-l-mips/>
-